

INF1103 Programming Fundamentals with DevOps
SME Green Grant Eligibility & Scope Compliance Auditor

1. Title Page

Team ID / team name: G2 / Group 2 (LAB-P10 - G2)
Repository: https://github.com/SYnthestic/INF1103_ProjectGrpWork
Date of submission: 14/10/2026

Student Name	Student ID	Email
Timothy Tan	2604749	2604749@sit.singaporetech.edu.sg
Yam Zhaomian Samuel	2603176	2603176@sit.singaporetech.edu.sg
Tia Isabel Solanki	2603925	2603925@sit.singaporetech.edu.sg
Toh Ming Yao	2604302	2604302@sit.singaporetech.edu.sg
Ng Zhi Bin	2603206	2603206@sit.singaporetech.edu.sg
Wu Kaiwen	2604750	2604750@sit.singaporetech.edu.sg

Table of Contents

1. Title Page 1
2. Introduction 3
3. Data Flow Diagram 4
4. Data Structure and Storage Design 5
5. Exception Handling Matrix 6

2. Introduction

Singapore's Green Plan 2030 is pushing SMEs to decarbonize their operations, and the government offers co-funding grants to help cover the cost. However, despite this support being available, 3 in 4 local SMEs remain unprepared to adopt sustainable practices, with over 70% not having accessed available government assistance.

Our application is a terminal tool that tells Singapore SME's whether their green upgrade proposal qualifies for government co-funding, and the type of grant they will qualify for. SMEs applying for schemes such as the Energy Efficiency Grant (EEG) Base/Advanced tier, Enterprise Development Grant (EDG) and the Sustainability Reporting Programme (SRP) face two problems. The rules are scattered across several schemes, and the proposals are written in free text. A consultant or grant officer has to search the various grants offered by the Singapore government, read each narrative, decide which greenhouse-gas scope (GHG Protocol Scope 1, 2 or 3) it affects, check the company against the SME definition, and work out the subsidy.

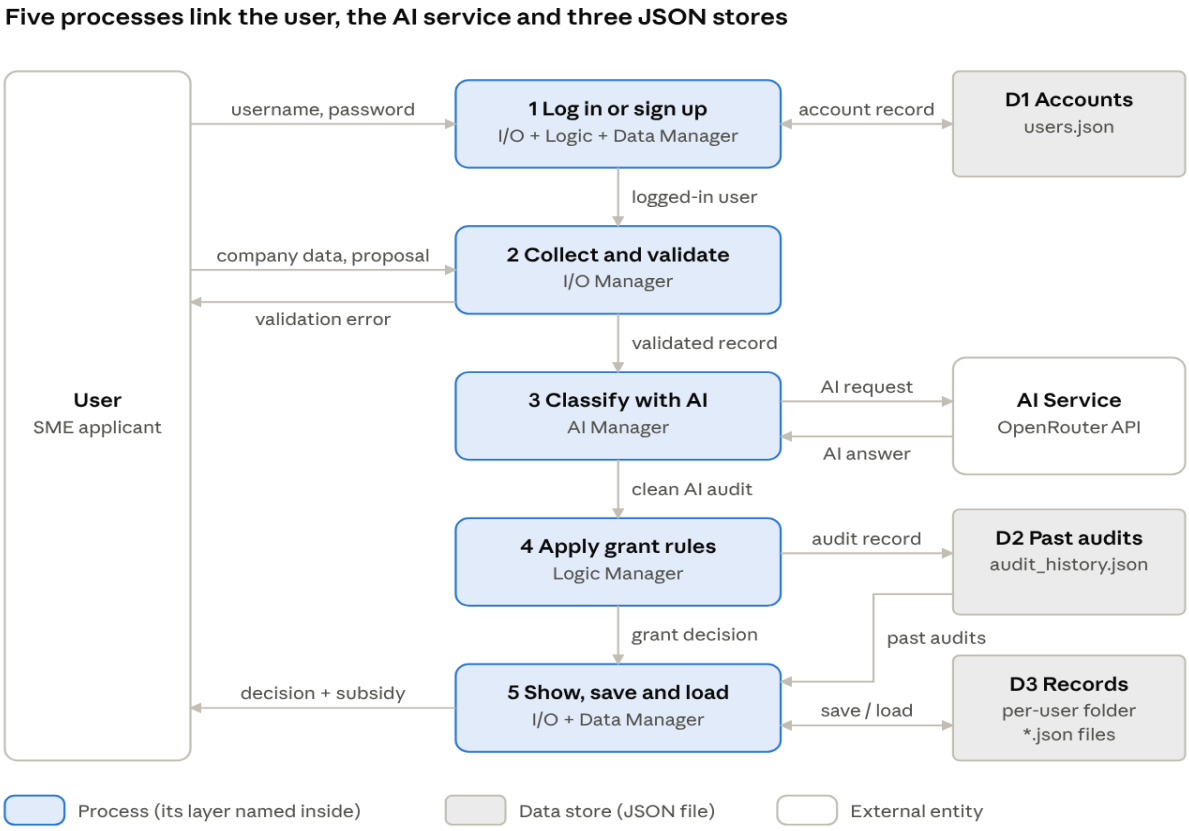
Using our auditor tool will help streamline the screening process for a quick eligibility result. A logged-in user enters the company's details and a free-text proposal. An AI model reads only the narrative and classifies it: GHG scope, estimated energy reduction, estimated lifetime abatement, and any prohibited items such as second-hand equipment. Fixed Python rules then make the decision. The AI never decides eligibility or calculates money, so the same inputs always give the same outcome. The user receives a decision (eg., *pre-approved*, *manual review* or *rejected*), the reasons, recommendations, and the subsidy amount. Every completed audit is saved to the user's own history, so said user cannot access another user's history.

The program uses four layers, each with one job and no classes: an I/O Manager (input, validation, screens), an AI Manager (the API call and cleaning of its answer), a Logic Manager (rules, subsidy maths, password hashing), and a Data Manager (JSON files). A small controller, main.py, passes data between them. The whole project runs from a Docker container, and an automated test suite and a GitHub Actions workflow check it on every push.

To lower the barrier for non-technical, first-time applicants, the system implements a guided Graphics User Interface (GUI). This visual design, including banners, menus, error graphics, and interactive button boundaries, is isolated within sme_interface_stores/sme_interface_gui.py. Containing zero print() or input() calls, this module strictly returns formatted string assets and coordinates. This keeps the I/O Manager as the exclusive user boundary, allowing complete aesthetic overhauls without modifying application logic. A lightweight controller, main.py, orchestrates data transitions across these functional layers. While an advanced TUI is not strictly necessary, it accelerates onboarding, reduces input friction, and simplifies operation for non-technical users. Finally, the entire ecosystem is containerised via Docker and continuously verified through an automated test suite within a GitHub Actions workflow on every push.

3. Data Flow Diagram

Data moves from the user through validation and the AI service to fixed grant rules, and only then to the three JSON stores or back to the screen.



Data flow diagram · 5 processes, 3 data stores, 2 external entities.

The layers are the processes: the I/O Manager owns processes 1, 2, and 5 (screens and checks), the AI Manager owns 3, the Logic Manager owns 4, and the Data Manager is the only code that reads or writes D1 to D3. `main.py` carries the data between them, so no layer calls another directly.

1. **Log in or sign up (process 1).** The I/O Manager collects the username and password. The Logic Manager hashes the password and compares it with the record in `users.json` (D1). Logging in switches the program to that user's own records folder.
2. **Collect and validate (process 2).** Company details, proposal type, narrative, and costs are checked field by field. An invalid value is rejected with a message and asked for again, so only validated data moves on.
3. **Classify with AI (process 3).** Only the proposal narrative is sent to the AI service. Its reply is cleaned into a fixed JSON structure (scope, estimated energy reduction, abatement, exclusion keywords). If the call fails or the answer is unusable, a safe default marked as failed is returned instead.
4. **Apply grant rules (process 4).** The Logic Manager checks exclusions, the SME definition, and the energy-reduction thresholds in a fixed priority order and calculates the subsidy. A completed audit is stored in `audit_history.json` (D2); a failed AI result is not.

5. **Show, save, and load (process 5).** The decision, reasons, and recommendations are displayed. The user can save or load records in their own folder (D3) and read their past audits from D2.

4. Data Structure and Storage Design

All data is stored as JSON text files in one data folder: `/app/data` in Docker (linked to a host folder with `-v`) or the working folder when run directly. Only the Data Manager reads or writes them.

File	Format	Structure of a stored record	Written when
`users.json`	JSON object keyed by username	username, algorithm, iterations, salt, password_hash, created_at. The password itself is never stored	An account is created (while logged out)
`audit_history.json`	JSON object: username to a list of audits	audit_id, timestamp, application (the 11 form fields: company name, industry, revenue, employees, local equity, proposal type, chosen grant, narrative, baseline spend, retrofit cost, advisory fee), ai_audit (scope, energy reduction %, abatement tonnes, exclusion keywords, bottleneck, reasoning, ai_status), result (is_sme, decision_status, reasons, recommendations, matched schemes, subsidy, audit_passed)	An audit completes with a usable AI answer
`records/<username>/<name>.json`	JSON list	The application dictionaries the user chose to save (the older nested layout is still readable) <name> is limited to 50 characters, enforced in such a way that the program will prevent you from writing >50 characters	The user saves (option 2)
`.env`	KEY=value text	OPENROUTER_API_KEY, optional DATA_DIR	Edited by hand, never committed (GitHub will refuse to push the KEY

Passwords are stored as salted PBKDF2-HMAC-SHA256 hashes (200,000 iterations) and compared in constant time. Each write goes to a `.tmp` file that is then swapped in with os.replace, so a crash never leaves half a file. A damaged file is renamed `*.corrupt` instead of being overwritten, and a `*.lock` file stops two copies of the app writing at once. Each user's saved files sit in a private folder, so one user cannot open another's.

5. Exception Handling Matrix

No single failure ends the session: each layer checks what it receives, falls back to a safe value or a clear message, and never saves a result it cannot trust.

Exception scenario	Possible cause	Detection method	Handling strategy	User outcome
AI connection failure	No network, service down, slow reply	Exception or 60 s timeout in the API thread	Fallback audit marked `failed`, decided as manual review; no automatic retry	Told the call failed and why; audit not saved; can re-run with option 7 (exclusively AI Manager)
Missing or wrong API key	No `.env`, key not passed to Docker, wrong `.env` key	`RuntimeError`, or 401 from the service	Same fallback as above	Message names `OPENROUTER_API_KEY`; rest of the app still works. But AI Manager will not run
Unusable AI answer	Not JSON, wrong types (`"30%"`), out-of-range values, `"None"` as a keyword	JSON parse and type checks in the AI Manager	JSON recovered from surrounding text; each field converted, clamped or defaulted; placeholders dropped	Decision on the cleaned values, or manual review if the scope is unclear; an unparseable answer is not saved
Invalid user input	Letters, negatives, NaN/inf, empty or over-long text, bad menu number	Validation loops and checks in the I/O Manager	Rejected and asked again; nothing passed on	Short error message, same prompt
Unusable financial data	Zero baseline spend or fee; subsidy above retrofit cost	Rule checks in the Logic Manager	Manual review with the reason, or subsidy capped at the retrofit cost	Decision says what to supply or why the amount was limited
Wrong password or unknown user	Typing error, guessing, forgot password	Hash comparison (a dummy hash runs for unknown users)	Login refused, no hint which part was wrong	Generic "login failed" due to “invalid username or password”

Exception scenario	Possible cause	Detection method	Handling strategy	User outcome
Damaged data file	Hand edit, interrupted copy (accounts, history, saved records)	JSON parse and shape check on read	Renamed *.corrupt and a fresh file started; if unreadable, writes are refused	Told the file is unreadable; old data kept in the backup; work continues
Unsafe file name	../x, illegal characters, bad username	Name and pattern checks in the Data Manager	Refused before any file is touched	"Invalid file name", asked again
Write fails or is interrupted	Crash, disk full, no permission	Exception around the write	Write to .tmp, then os.replace; temp file removed	Told it could not be saved; old file intact
Two copies writing at once	Two containers share one data folder	.lock file created with O_EXCL; 5 s wait; stale after 30s	Second writer waits, then gives up	"Could not save"; retrying works
Unexpected error in the AI or logic step	Unforeseen bug or data shape	try/except around each step in main.py	Error reported; program continues	Short message, back to the menu
Ctrl+C or no keyboard input	User quits; docker run without -it	KeyboardInterrupt / EOFError caught by a global handler	Clean exit	"Interrupted. Goodbye!" or a hint to use docker run -it